

IPOG — Instituto de Pós-Graduação e Graduação

Projeto Integrador • Área 03 — Banco de Dados

Modelagem e análise de dados de vendas

Northwind Traders • PostgreSQL e MongoDB

Entrega 2

Compreensão dos dados e modelagem documental

Giovanne Espindola

Trabalho individual

Setembro de 2026

1. Introdução e objetivo

A Northwind Traders é uma empresa fictícia de distribuição de alimentos usada em uma base didática de vendas. Seus registros permitem estudar clientes, produtos, pedidos, fornecedores e a organização da equipe comercial. Este projeto utiliza essa base para desenvolver e comparar uma solução relacional em PostgreSQL e uma solução orientada a documentos em MongoDB.

O objetivo de negócio é identificar como o valor dos pedidos se distribui entre produtos, categorias, clientes e períodos, além de avaliar informações de estoque e de envio. O objetivo técnico é representar esse domínio nos dois bancos, executar consultas equivalentes e analisar os resultados, a organização dos dados e o desempenho observado.

O trabalho reúne a compreensão do negócio, a análise exploratória, a avaliação da qualidade dos dados e a modelagem documental. A implementação no MongoDB contempla a organização das coleções, a transformação dos registros relacionais, os índices e a conferência da integridade dos dados.

1.1 Orientação metodológica

O CRISP-DM organiza projetos de dados em seis fases relacionadas: compreensão do negócio, compreensão dos dados, preparação, modelagem, avaliação e implantação [1]. A aplicação dessa metodologia abrangeu a compreensão do domínio, a avaliação da qualidade da base, a preparação dos dados e a definição das estruturas de armazenamento.

Fase	Aplicação no projeto
Compreensão do negócio	Definir o contexto, as perguntas e os limites da análise.
Compreensão dos dados	Examinar tabelas, relacionamentos, valores, ausências e consistência.
Preparação dos dados	Ajustar tipos e regras e preparar a transformação para documentos.
Modelagem	Desenvolver os modelos relacional e documental.
Avaliação	Conferir a equivalência das consultas e medir o desempenho.
Implantação	Disponibilizar as estruturas de armazenamento e documentar os resultados.

O uso é adaptado a um projeto de banco de dados: a modelagem será de estruturas de armazenamento, sem treinamento de modelos estatísticos. As fases podem ser retomadas. Por exemplo, a ausência de custos de aquisição levou a limitar as perguntas financeiras ao valor dos pedidos, sem calcular lucro.

1.2 Base e ferramentas

Foi utilizada a adaptação do Northwind para PostgreSQL mantida no repositório [pthom/northwind_psql](#) [2], cuja cópia SQL está preservada no projeto. A análise usa PostgreSQL 16, consultas SQL e um notebook Jupyter em Python. O PostgreSQL permite trabalhar com relacionamentos, restrições e valores decimais, além de inspecionar os planos das consultas.

O MongoDB 7 foi escolhido como o banco documental previsto no projeto da disciplina [3]. A transformação dos registros foi realizada em SQL, e a carga documental utilizou o mongosh, ambiente de execução de comandos do MongoDB.

2. Compreensão do negócio

A empresa comercializa produtos de diferentes categorias, fornecidos por empresas cadastradas. Os clientes fazem pedidos registrados por funcionários. Cada pedido reúne um ou mais itens, com produto, quantidade, preço praticado e desconto. Uma transportadora é indicada no pedido. A equipe comercial também está vinculada a territórios agrupados em regiões.

A base representa o cadastro comercial, os pedidos e informações de envio. Ela não registra o recebimento de pagamento nem a confirmação de entrega ao cliente. Assim, o processo observado vai do registro do pedido às informações disponíveis sobre seu envio.

2.1 Informações que se pretende obter

Pergunta de negócio	Uso da informação
Quais categorias e produtos concentram o valor dos pedidos?	Identificar a participação de cada parte do catálogo.
Como variam o volume e o valor médio dos pedidos ao longo dos meses?	Acompanhar a evolução comercial dentro do período disponível.
Quais clientes compram mais e quais estão sem compras recentes?	Apoiar ações de relacionamento com clientes.
Quanto foi registrado por vendedor e por equipe?	Descrever a distribuição das vendas entre os responsáveis.
Quais produtos estão abaixo do ponto de reposição?	Identificar sinais de atenção no estoque cadastrado.
Qual é o intervalo entre o pedido e o envio por transportadora?	Examinar o tempo até o envio registrado, sem confundir-lo com prazo de entrega.

Essas perguntas orientam o detalhamento das consultas nas próximas etapas. Nesta entrega, a exploração já verifica o período, o volume, a distribuição dos valores e a participação das categorias.

2.2 Definições para interpretar os resultados

Valor do item: preço praticado no item $\times$ quantidade $\times$ (1 – desconto). O valor de um pedido é a soma de seus itens. Neste relatório, “valor dos pedidos” é o total após descontos, sem acrescentar frete. Ele não comprova recebimento financeiro. Os valores são apresentados em unidades monetárias da base, sem atribuir uma moeda que não está identificada nas tabelas analisadas.

Ticket médio: soma do valor dos pedidos dividida pelo número de pedidos. O cálculo agrupa os itens antes de calcular a média, para não tratar cada item como um pedido independente.

Recência: as análises futuras usarão como referência o último dia com pedido, 06/05/1998. Usar a data atual tornaria todos os clientes antigos e não ajudaria a distinguir seu comportamento no recorte.

Limites do negócio: não há custo de aquisição para calcular margem ou lucro, nem registros de pagamentos, devoluções ou cancelamentos. A ausência dessas tabelas não comprova que esses eventos nunca ocorreram. A segmentação demográfica não é viável com as tabelas vazias, mas a segmentação por recência, frequência e valor das compras continua possível.

3. Modelo conceitual

O diagrama apresenta 11 entidades do recorte de vendas. Os identificadores descrevem como cada entidade é reconhecida; tipos SQL, índices e detalhes de armazenamento ficam para o modelo lógico e físico. ITEM DO PEDIDO e ATUAÇÃO são entidades associativas.

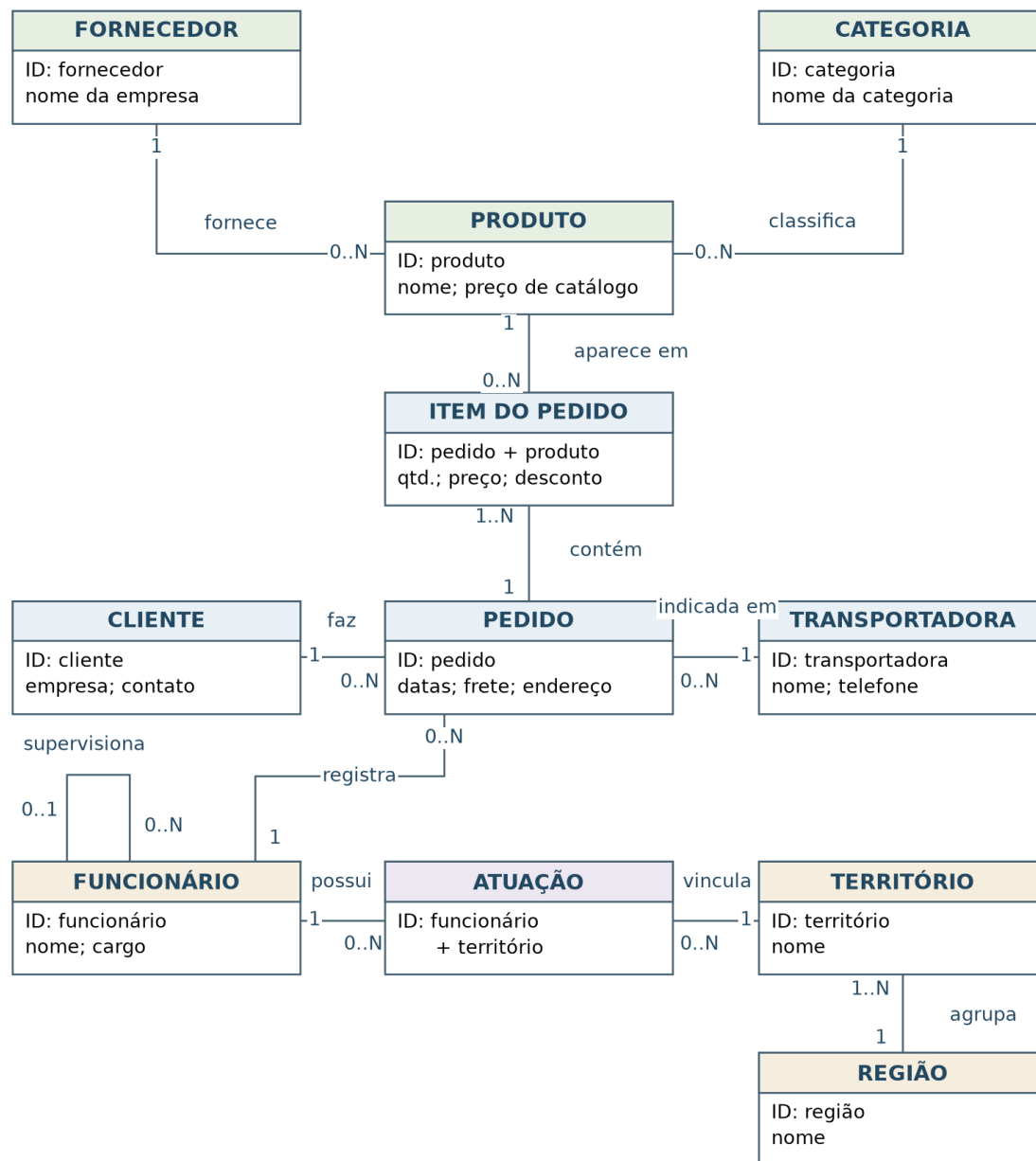


Figura 1 — Entidades e relacionamentos do recorte Northwind. Elaboração própria a partir da base [2]. Os números próximos a cada entidade indicam quantas ocorrências daquele lado podem se relacionar com uma ocorrência do outro lado: 1 = exatamente uma; 0..1 = opcional; 0..N = nenhuma ou várias; 1..N = uma ou várias.

3. Modelo conceitual — regras adotadas

As regras abaixo são escolhas para este projeto, compatíveis com os dados observados. A ausência de um caso na amostra não significa que ele deva ser proibido no modelo.

Relacionamento	Regra adotada
Cliente e pedido	Cada pedido tem um cliente. Um cliente pode ter zero ou vários pedidos.
Funcionário e pedido	Cada pedido tem um funcionário responsável. Um funcionário pode não ter pedidos.
Transportadora e pedido	Cada pedido tem uma transportadora indicada. Ela pode não aparecer em nenhum pedido.
Pedido e item	Um pedido tem um ou vários itens. Cada item pertence a um único pedido.
Produto e item	Cada item se refere a um produto. Um produto pode não aparecer em pedidos.
Categoria e produto	Cada produto pertence a uma categoria. A categoria pode não ter produtos.
Fornecedor e produto	Cada produto tem um fornecedor cadastrado. O fornecedor pode não ter produtos.
Região e território	Cada território pertence a uma região. Neste recorte, uma região reúne um ou vários territórios.
Funcionário, atuação e território	Cada atuação vincula um funcionário a um território. Ambos podem ter zero ou vários vínculos.
Supervisão de funcionários	Um funcionário tem no máximo um superior e pode supervisionar vários funcionários.

3.1 Identificação e participação

ITEM DO PEDIDO é identificado pelo par **pedido e produto**. O modelo não permite repetir esse par; a quantidade registra quantas unidades foram incluídas. ATUAÇÃO é identificada pelo par **funcionário e território**, permitindo representar vários responsáveis por um território quando necessário.

Na amostra, os 49 vínculos de atuação abrangem 49 territórios distintos: nenhum tem dois responsáveis registrados. A estrutura N:N é mantida como possibilidade do modelo. Também existem dois clientes sem pedidos, três transportadoras sem pedidos e quatro territórios sem funcionário vinculado.

As ligações de pedido com cliente, funcionário e transportadora, e de produto com categoria e fornecedor, aceitam nulos na fonte, embora não apresentem nulos na amostra. Sua obrigatoriedade é uma regra adotada no modelo do projeto. Já a exigência de ao menos um item por pedido precisa de validação adicional: uma chave estrangeira, sozinha, não garante que o pedido tenha itens.

3.2 Recorte do modelo

As tabelas **customer_demographics** e **customer_customer_demo** estão vazias e não serão usadas nas análises previstas. **us_states** tem 51 registros, mas não participa das relações utilizadas. As três permanecem preservadas na fonte. Sua exclusão se deve ao escopo; uma entidade pode ser válida mesmo sem registros.

4. Análise exploratória e qualidade dos dados

4.1 Procedimento e inventário

A conferência foi realizada em 05/09/2026, diretamente no PostgreSQL, em uma transação somente leitura. O notebook existente foi revisado e seus resultados estruturais foram conferidos por consultas independentes: contagem exata de linhas, inspeção das 92 colunas, nulos, chaves, relações e faixas de valores. Os comandos selecionados e suas saídas aparecem ao longo desta seção.

A análise usa o schema **public**, que preserva a fonte. O schema **nw** contém a implementação relacional já iniciada, com 11 tabelas e 3.311 linhas. Esse schema foi utilizado como origem da transformação documental. Separar os schemas permite comparar a origem com o modelo proposto.

Tabela	Linhas	Colunas
categories	8	4
customer_customer_demo	0	2
customer_demographics	0	2
customers	91	11
employee_territories	49	2
employees	9	18
order_details	2155	5
orders	830	14
products	77	10
region	4	2
shippers	6	3
suppliers	29	12
territories	53	3
us_states	51	4

A soma é **3.362 linhas em 14 tabelas**. Esse número reúne tipos diferentes de registro; não corresponde ao número de vendas. Há 830 pedidos e 2.155 itens, com média de 2,60 itens distintos por pedido e intervalo de 1 a 25 itens.

Consulta 1 — Período e clientes com pedido.

```
SELECT min(order_date) AS primeiro_pedido,
       max(order_date) AS ultimo_pedido,
       count(*) AS pedidos,
       count(DISTINCT customer_id) AS clientes_com_pedido
FROM public.orders;
```

Saída da consulta:

primeiro_pedido	ultimo_pedido	pedidos	clientes_com_pedido
1996-07-04	1998-05-06	830	89

O recorte vai de 04/07/1996 a 06/05/1998. Dos 91 clientes cadastrados, 89 aparecem em pedidos. Os demais podem ser estudados como clientes sem compra registrada nesse período.

4. Análise exploratória — valores e concentração

4.2 Valor e distribuição dos pedidos

Consulta 2 — Total e distribuição por pedido. Os campos são convertidos para decimal antes da multiplicação. O arredondamento é feito na apresentação do resultado, após a soma.

```
WITH totais AS (
  SELECT order_id,
         sum(unit_price::numeric * quantity *
              (1 - discount::numeric)) AS valor
  FROM public.order_details GROUP BY order_id
)
SELECT round(sum(valor), 2) AS total,
       round(avg(valor), 2) AS media,
       round(percentile_cont(0.5) WITHIN GROUP
              (ORDER BY valor)::numeric, 2) AS mediana,
       round(min(valor), 2) AS minimo,
       round(max(valor), 2) AS maximo
FROM totais;
```

Saída da consulta:

total	media	mediana	minimo	maximo
1265793.04	1525.05	943.25	12.50	16387.50

O total é **1.265.793,04**, com ticket médio de **1.525,05** e mediana de **943,25**. A média maior que a mediana e o maior pedido de 16.387,50 indicam a influência de pedidos de valor elevado. Por isso, a mediana complementa a média na descrição do pedido típico.

4.3 Participação das categorias

Consulta 3 — Valor agrupado pela categoria do produto.

```
SELECT c.category_name AS categoria,
       round(sum(d.unit_price::numeric * d.quantity *
                 (1 - d.discount::numeric)), 2) AS valor
FROM public.order_details d
JOIN public.products p ON p.product_id = d.product_id
JOIN public.categories c ON c.category_id = p.category_id
GROUP BY c.category_name ORDER BY valor DESC;
```

Saída da consulta:

categoria	valor
Beverages	267868.18
Dairy Products	234507.29
Confections	167357.23
Meat/Poultry	163022.36
Seafood	131261.74
Condiments	106047.09
Produce	99984.58
Grains/Cereals	95744.59

Beverages e Dairy Products somam **502.375,47**, aproximadamente **39,7%** do valor total. Essa concentração ajuda a identificar a participação comercial das categorias. Ela não mede lucro, pois os custos dos produtos não estão disponíveis.

4. Análise exploratória — período e dados ausentes

4.4 Distribuição temporal

ano	pedidos	valor
1996	152	208083.97
1997	408	617085.20
1998	270	440623.87

Os pedidos foram agrupados pelo ano de `order_date`, com contagem dos pedidos e soma dos valores de seus itens. **1996 e 1998 são anos parciais**: os registros de 1996 começam em julho; os de 1998 terminam em maio. Comparar seus totais com 1997 como se fossem anos completos produziria uma interpretação inadequada de crescimento. Análises futuras usarão meses comparáveis; a janela curta limita conclusões sobre sazonalidade.

4.5 Completude

Foram contados os nulos de cada coluna por meio de `count(*) - count(coluna)`. Onze das 92 colunas apresentam pelo menos um nulo. As tabelas vazias não permitem avaliar completude de registros.

Coluna	Nulos	Total	%
customers.region	60	91	65.93
customers.postal_code	1	91	1.10
customers.fax	22	91	24.18
employees.region	4	9	44.44
employees.reports_to	1	9	11.11
orders.shipped_date	21	830	2.53
orders.ship_region	507	830	61.08
orders.ship_postal_code	19	830	2.29
suppliers.region	20	29	68.97
suppliers.fax	16	29	55.17
suppliers.homepage	24	29	82.76

Os nulos se concentram em informações de contato e endereço. Sem documentação adicional, não é possível distinguir automaticamente ausência de preenchimento de campo não aplicável. Não serão substituídos por valores inventados. O nulo em **employees.reports_to** representa o funcionário sem superior cadastrado, no topo da hierarquia observada.

Consulta 4 — Data de envio e intervalo até o envio.

```
SELECT count(*) AS pedidos,
       count(*) FILTER (WHERE shipped_date IS NULL) AS sem_data,
       round(100.0 * count(*) FILTER
         (WHERE shipped_date IS NULL) / count(*), 2) AS pct_sem_data,
       round(avg(shipped_date - order_date), 2) AS dias_ate_envio
FROM public.orders;
```

Saída da consulta:

pedidos	sem_data	pct_sem_data	dias_ate_envio
830	21	2.53	8.49

Há **21 pedidos sem data de envio registrada (2,53%)**. A média de 8,49 dias considera apenas os 809 pedidos com data de envio. Nenhum desses números informa quando o cliente recebeu a mercadoria ou comprova que os pedidos sem data nunca foram enviados.

4. Análise exploratória — integridade e validade

4.6 Identificadores e relacionamentos

A fonte possui **14 chaves primárias** e **13 chaves estrangeiras**. A revisão não encontrou duplicidade de identificadores nas 14 tabelas nem referências órfãs nas 13 relações verificadas. Isso não descarta duplicidades de cadastro com identificadores diferentes, que exigiriam critérios adicionais de comparação de nomes e endereços.

Consulta 5 — Exemplos de verificação de relacionamentos.

```
SELECT 'pedido sem cliente válido' AS verificacao, count(*) AS casos
FROM public.orders o WHERE NOT EXISTS (
  SELECT 1 FROM public.customers c WHERE c.customer_id = o.customer_id)
UNION ALL
SELECT 'item sem pedido válido', count(*)
FROM public.order_details d WHERE NOT EXISTS (
  SELECT 1 FROM public.orders o WHERE o.order_id = d.order_id)
UNION ALL
SELECT 'pedido sem item', count(*)
FROM public.orders o WHERE NOT EXISTS (
  SELECT 1 FROM public.order_details d WHERE d.order_id = o.order_id);
```

Saída da consulta:

verificacao	casos
pedido sem cliente válido	0
item sem pedido válido	0
pedido sem item	0

Os três testes retornaram zero. A presença de itens em todos os pedidos também permite calcular o ticket usando os 830 pedidos da base, sem excluir pedidos vazios.

4.7 Faixas de valores e coerência temporal

Consulta 6 — Valores fora das faixas adotadas e envio anterior ao pedido.

```
SELECT 'quantidade <= 0' AS verificacao, count(*) AS casos
FROM public.order_details WHERE quantity <= 0
UNION ALL
SELECT 'preço < 0', count(*) FROM public.order_details WHERE unit_price < 0
UNION ALL
SELECT 'desconto fora de [0, 1)', count(*)
FROM public.order_details WHERE discount < 0 OR discount >= 1
UNION ALL
SELECT 'envio anterior ao pedido', count(*)
FROM public.orders WHERE shipped_date < order_date;
```

Saída da consulta:

verificacao	casos
quantidade <= 0	0
preço < 0	0
desconto fora de [0, 1)	0
envio anterior ao pedido	0

Os preços dos itens variam de **2,00 a 263,50**, as quantidades de **1 a 130** e os descontos de **0% a 25%**. Não foram encontrados casos fora das regras testadas. O desconto inferior a 100% é uma regra deste projeto; o máximo observado de 25% não será transformado em limite obrigatório.

A fonte não possui restrições **CHECK**, apesar de já ter chaves e outras regras de integridade. O modelo proposto usa validações explícitas para impedir valores inválidos em novas cargas. Ausência de erro na amostra e existência de uma proteção no banco são verificações diferentes.

4. Análise exploratória — precisão e histórico

4.8 Valores decimais e arredondamento

Na fonte, preço e desconto dos itens são armazenados como **real**, um tipo aproximado. A documentação do PostgreSQL recomenda **numeric** para valores monetários que exigem exatidão [4]. A conversão permite calcular sobre valores decimais, mas não recupera uma precisão que a origem já tenha perdido.

A soma executada com ponto flutuante foi **1265793.038653364**; com conversão para decimal, **1265793.0395**. Ambas são apresentadas como 1.265.793,04 com duas casas decimais. Há outra diferença, independente do tipo: o momento do arredondamento.

Consulta 7 — Arredondar a soma ou arredondar cada item.

```
SELECT round(sum(unit_price::numeric * quantity *
               (1 - discount::numeric)), 2) AS arredondar_total,
       sum(round(unit_price::numeric * quantity *
               (1 - discount::numeric), 2)) AS arredondar_itens
FROM public.order_details;
```

Saída da consulta:

arredondar_total	arredondar_itens
1265793.04	1265793.29

A diferença é **0,25**. Este relatório usa **arredondamento apenas no total**. As views já criadas no schema nw arredondam por item; por isso, a comparação futura precisa adotar a mesma regra nos dois bancos antes de avaliar equivalência. O valor maior não indica, por si só, perda de dados na migração.

4.9 Preço e endereço registrados no pedido

Consulta 8 — Comparação do preço do item com o cadastro do produto.

```
SELECT count(*) AS itens,
       count(*) FILTER
         (WHERE d.unit_price <> p.unit_price) AS precos_diferentes
FROM public.order_details d
JOIN public.products p ON p.product_id = d.product_id;
```

Saída da consulta:

itens	precos_diferentes
2155	662

Em **662 dos 2.155 itens (30,7%)**, o preço praticado difere do preço do catálogo. A diferença não comprova quando ou por que o preço mudou. Ela mostra por que o cálculo deve usar o preço do item, preservado no pedido, em vez de substituí-lo pelo preço cadastrado do produto.

O mesmo cuidado se aplica ao endereço. Ao comparar destinatário, endereço, cidade, região, CEP e país, os seis campos coincidem com o cadastro do cliente em **748 dos 830 pedidos**; em 82 há diferença. A igualdade foi comparada tratando nulos correspondentes como iguais. O endereço do pedido será preservado como informação daquele pedido, sem eliminar campos apenas porque repetem o cadastro atual.

4.10 Avaliação da base

A base é adequada às análises descritivas propostas: os relacionamentos conferem e não há violações nas faixas testadas. Os cuidados principais são tratar nulos conforme seu significado, manter os dados próprios do pedido, padronizar decimais e arredondamento e respeitar as limitações temporais e de cobertura do negócio.

5. Abordagem híbrida

O fluxo implementado é **PostgreSQL public → PostgreSQL nw → MongoDB**. A fonte original permanece preservada. O schema nw aplica as decisões relacionais; o MongoDB recebeu uma cópia para consultas, carregada por um processo em lote. A atualização documental foi definida em um único sentido, tendo o modelo relacional como origem.

5.1 Organização do MongoDB

Coleção	Organização implementada
orders	Um documento por pedido, com itens embutidos, datas, frete, endereço de envio e identificadores de cliente, funcionário e transportadora. Nos itens: produto, categoria, quantidade, preço praticado e desconto.
products	Produto com estoque, categoria e fornecedor identificados; nomes selecionados foram copiados para facilitar a leitura.
customers	Cadastro do cliente com endereço agrupado em um objeto.
employees	Cadastro do funcionário, referência ao superior e lista de territórios de atuação.
categories, suppliers e shippers	Cadastros completos para preservar também registros sem pedido ou produto associado.
territories e regions	Cadastros completos, incluindo territórios sem funcionário vinculado.

Foram implementadas **nove coleções**. Os 830 pedidos e seus 2.155 itens formam 830 documentos em orders. O critério de agrupamento segue a orientação de considerar o acesso aos dados ao decidir entre incorporação e referências [5].

5.2 Carga e validação

O processo usa identificadores estáveis e substituição por identificador, evitando duplicações ao repetir a carga. Registros removidos da origem provocam interrupção para revisão antes da gravação; não há exclusão automática. Valores monetários são convertidos de **numeric para Decimal128**, sem passar por float; datas e nulos seguem uma convenção única [6].

Os dados copiados de cadastros representam o estado disponível na extração, sem serem apresentados como histórico da data da venda. O preço praticado e o endereço de envio vêm do próprio pedido. A duplicação de nomes e categorias exige atualização das cópias quando a fonte mudar.

A validação compara identificadores, contagens, quantidade de itens por pedido e valores calculados com a mesma regra. Foram definidos validadores de estrutura e índices conforme as consultas. Referências entre coleções são conferidas após a carga; a validação de documentos não substitui as chaves estrangeiras relacionais [7].

5.3 Comparação prevista e estado atual

A hipótese é que incorporar itens simplifique a leitura de um pedido completo, enquanto consultas que cruzam cadastros possam exigir referências e agregações adicionais. O resultado de desempenho está em aberto. A medição usará consultas equivalentes, mesma máquina, recursos registrados, aquecimento e pelo menos dez repetições, com mediana e dispersão dos tempos.

O MongoDB está carregado com **1.107 documentos**. Seu modo atual é **standalone**, sem transações entre vários documentos; a escrita em um único documento é atômica [5, 8]. A transformação SQL, a carga mongosh, os validadores e os índices são detalhados nas seções seguintes. As consultas analíticas equivalentes permanecem para as próximas etapas. O volume pequeno não permite concluir sobre escalabilidade.

6. Critérios para a modelagem documental

A exploração identificou uma base com 830 pedidos, 2.155 itens, 91 clientes e 77 produtos. Os pedidos cobrem parte de 1996, todo o ano de 1997 e parte de 1998. Seus valores apresentam concentração em determinadas categorias e diferença entre média e mediana, o que orienta o aprofundamento das análises comerciais.

As verificações de identificadores, referências e faixas de valores não apontaram inconsistências nas regras examinadas. As principais limitações são as datas de envio ausentes, a falta de custos e pagamentos, os anos parciais e o uso de tipos aproximados na fonte. Esses achados fundamentaram os critérios adotados na transformação dos dados.

6.1 Preservação das informações da venda

Os preços praticados e os endereços de envio foram mantidos como atributos próprios dos pedidos. A diferença entre o preço do item e o preço de catálogo em 662 itens mostrou que o cadastro atual não poderia substituir a informação da venda. Da mesma forma, as diferenças de endereço em 82 pedidos justificaram conservar o endereço de envio integralmente.

Os dados ausentes permaneceram nulos. A ausência de data de envio, por exemplo, não foi convertida em uma data estimada nem interpretada como comprovação de que o pedido nunca foi enviado. Os cadastros sem movimento também foram preservados, permitindo distinguir ausência de compra registrada de ausência do cliente na base.

6.2 Consistência dos indicadores

O cálculo dos valores manteve o preço praticado no item, a quantidade e o desconto, com arredondamento somente após a soma. A conversão para tipos decimais evitou introduzir novas aproximações de ponto flutuante na transformação, respeitando a precisão disponível na origem.

A conferência do modelo documental foi orientada pela comparação de identificadores e conteúdo, além das contagens. Esse critério permite verificar se a reorganização das linhas em objetos e arrays preservou as informações utilizadas nos indicadores. A igualdade de contagens, isoladamente, não comprovaria a manutenção de preços, descontos, datas ou endereços.

As regras de qualidade da base foram consideradas na definição dos validadores e na conferência das referências. A análise das consultas e do desempenho depende dessa consistência: diferenças decorrentes de arredondamento ou de perda de dados não devem ser interpretadas como características dos modelos de armazenamento.

7. Modelagem documental

7.1 Organização das coleções

A modelagem documental foi desenvolvida a partir das 11 tabelas do schema **nw**, preservando seus atributos, identificadores e relacionamentos. Os dados foram reorganizados em nove coleções no MongoDB. O principal agrupamento ocorreu entre pedidos e itens: o cabeçalho e os produtos vendidos passaram a compor um único documento por pedido.

Coleção	Documentos	Organização dos dados
orders	830	Pedido com datas, frete, referências a cliente, funcionário e transportadora, endereço shipping e array items.
products	77	Produto com preço de catálogo, estoque, categoria e fornecedor identificados e nomeados.
customers	91	Cadastro do cliente com informações de contato e endereço agrupado em objeto.
employees	9	Cadastro do funcionário, endereço, referência ao superior e array de territórios.
categories	8	Identificador, nome e descrição da categoria.
suppliers	29	Cadastro do fornecedor com contato e endereço agrupado em objeto.
shippers	6	Identificador, nome e telefone da transportadora.
territories	53	Identificador e nome do território, com referência à região.
regions	4	Identificador e nome da região comercial.

Foram obtidos **1.107 documentos**. A diferença em relação às 3.311 linhas relacionais decorre da incorporação dos 2.155 itens nos pedidos e dos 49 vínculos de território nos funcionários. Os cadastros sem movimento foram mantidos, incluindo clientes sem compra e transportadoras sem pedido associado.

7.2 Incorporação e referências

Os itens foram incorporados porque pertencem a um pedido e são consultados junto com ele. O endereço de envio também foi agrupado no pedido, preservando a informação registrada na venda. Essa decisão considera as diferenças de endereço identificadas na análise exploratória e evita substituí-lo pelo cadastro atual do cliente.

Produtos, clientes, funcionários e transportadoras permaneceram em coleções próprias, pois participam de vários pedidos e possuem atributos independentes de uma venda. Seus identificadores estabelecem as referências entre documentos. A organização combina incorporação e referências conforme as formas de acesso aos dados [5].

Nos funcionários, o array territories reúne os territórios de atuação, enquanto reports_to conserva a referência ao superior. As coleções territories e regions preservam a malha comercial completa, incluindo territórios sem funcionário vinculado. Os pedidos foram mantidos em coleção própria, evitando um array crescente de compras no cadastro de cada cliente.

7. Modelagem documental — estrutura do pedido

7.3 Representação do pedido 10248

O pedido 10248 pertence ao cliente VINET e contém três produtos. No modelo relacional, seus dados estão distribuídos entre `orders` e `order_items`. No MongoDB, o endereço de envio e os itens estão incorporados ao documento do pedido. O trecho abaixo apresenta os campos principais e o primeiro item; os demais itens foram omitidos somente desta representação.

```
{
  _id: 10248,
  customer_id: "VINET",
  order_date: ISODate("1996-07-04T00:00:00Z"),
  freight: Decimal128("32.38"),
  shipping: {
    name: "Vins et alcools Chevalier",
    street: "59 rue de l'Abbaye",
    city: "Reims", region: null,
    postal_code: "51100", country: "France"
  },
  items: [{
    product: {
      _id: 11, name: "Queso Cabrales",
      category: { _id: 4, name: "Dairy Products" }
    },
    unit_price: Decimal128("14.00"),
    quantity: 12, discount: Decimal128("0.000")
  }]
}
```

Os três produtos são 11, 42 e 72, com quantidades 12, 10 e 5 e preços praticados de 14,00, 9,80 e 34,80. O valor dos itens é **440,00**, sem incluir o frete de 32,38. Os preços foram obtidos dos itens da venda, preservando a diferença em relação ao preço de catálogo.

O nome do produto e sua categoria foram copiados para os itens a partir dos cadastros disponíveis na extração. Essas informações facilitam a leitura do pedido, mas não constituem histórico do catálogo na data da venda. Uma atualização cadastral exige nova carga para atualizar as cópias.

7.4 Tipos e valores ausentes

Os identificadores numéricos e as quantidades foram armazenados como **Int32**. Os códigos de clientes e territórios permaneceram como **strings**, preservando possíveis zeros à esquerda. Preços, descontos e frete foram armazenados como **Decimal128**, e valores SQL NULL foram mantidos como **null**.

As datas foram convertidas para **JSON Date à meia-noite UTC**, conservando o dia registrado na origem. Esse horário é uma convenção de representação, pois o tipo `date` do PostgreSQL não registra a hora do evento. Campos opcionais, como região, CEP e data de envio, permaneceram presentes nos documentos, admitindo **null** conforme a origem.

8. Transformação e carga dos dados

8.1 Agrupamento dos registros relacionais

A extração foi realizada no PostgreSQL em uma transação **REPEATABLE READ**, mantendo a mesma visão da origem durante a leitura das coleções. Os documentos foram construídos com funções JSON, sem alterações nos registros de nw. A operação central consistiu em agrupar as linhas de order_items pelo identificador do pedido.

Consulta 9 — Agrupamento dos itens do pedido 10248.

```
SELECT o.order_id,
       jsonb_agg(jsonb_build_object(
         'product_id', i.product_id,
         'unit_price', i.unit_price,
         'quantity', i.quantity,
         'discount', i.discount
       ) ORDER BY i.product_id) AS items
FROM nw.orders o
JOIN nw.order_items i USING (order_id)
WHERE o.order_id = 10248
GROUP BY o.order_id;
```

A consulta retorna uma linha para o pedido, com seus três itens no array items. O conteúdo obtido é apresentado abaixo, com quebras de linha para facilitar a leitura.

```
{
  "order_id": 10248,
  "items": [
    {"product_id": 11, "unit_price": 14.00,
     "quantity": 12, "discount": 0.000},
    {"product_id": 42, "unit_price": 9.80,
     "quantity": 10, "discount": 0.000},
    {"product_id": 72, "unit_price": 34.80,
     "quantity": 5, "discount": 0.000}
  ]
}
```

A função **jsonb_build_object** formou os campos de cada item, e **jsonb_agg** reuniu os registros em um array ordenado por produto [12]. Na transformação completa, cada pedido recebeu também suas datas, frete, referências e endereço de envio. Os itens receberam o identificador e o nome do produto, além da categoria aninhada.

O mesmo princípio foi aplicado aos vínculos de território: os registros da relação entre funcionário e território foram agrupados por employee_id e incorporados ao funcionário. Os demais cadastros deram origem a documentos individuais em suas respectivas coleções.

8. Transformação — tipos e gravação

8.2 Preservação dos tipos BSON

O transporte utilizou **Extended JSON**, que permite representar tipos BSON não diferenciados pelo JSON comum [9]. Os valores monetários foram convertidos de numeric para uma representação decimal textual e interpretados como Decimal128 no MongoDB, sem passagem por ponto flutuante.

O preço 14,00 foi representado da seguinte forma:

```
{"$numberDecimal": "14.00"}
```

A leitura com **EJSON.parse**, configurada com `relaxed: false`, converteu os marcadores de decimais, inteiros e datas nos respectivos tipos BSON. As funções JSON do PostgreSQL preservaram os caracteres de nomes e endereços, incluindo apóstrofes, e mantiveram os valores nulos.

8.3 Gravação por identificador

A carga foi executada pelo mongosh, utilizando operações de substituição agrupadas em `bulkWrite`. Cada documento conservou o identificador da origem no campo `_id`. O comando abaixo apresenta a operação aplicada aos pedidos, em que documento representa o registro completo produzido na transformação.

```
db.orders.replaceOne(
  { _id: documento._id },
  documento,
  { upsert: true }
);
```

A opção **upsert** insere o documento quando o identificador ainda não existe. Quando existe, **replaceOne** substitui seu conteúdo, incluindo os objetos e arrays aninhados. Essa estratégia permite repetir a carga sem criar novos identificadores para os mesmos registros.

Execução	Documentos inseridos	Documentos modificados
Primeira carga	1.107	0
Repetição da mesma carga	0	0

O resultado da segunda execução confirmou a estabilidade da carga para o mesmo conjunto de dados. Alterações na origem exigem nova extração. A atualização é feita em lote, e identificadores existentes no destino que não aparecem na extração provocam interrupção antes das escritas, para revisão. Não foi adotada propagação automática de exclusões.

8.4 Limites da carga

O MongoDB foi utilizado em modo **standalone**, com atomicidade por documento, mas sem transação única envolvendo todas as coleções [8]. Uma interrupção durante a gravação pode deixar o lote incompleto. A recuperação adotada consiste na repetição integral da carga após corrigir a causa, seguida de nova conferência de integridade.

A rotina foi desenvolvida para o conjunto pequeno e estático do Northwind. Os documentos são lidos em memória, e coleções vazias são recusadas para revisão. A implementação não foi avaliada como mecanismo de sincronização contínua ou de migração de grandes volumes.

8. Validação da estrutura documental

8.5 Regras implementadas

As nove coleções receberam validadores de estrutura, aplicados também aos objetos e arrays aninhados. Foram definidos os campos obrigatórios, os tipos BSON e as faixas de valores. A validação foi configurada para rejeitar inserções e alterações incompatíveis com as regras [7].

Campo ou grupo	Regra adotada
Preços e frete	Decimal128, com valor maior ou igual a zero.
Quantidade do item	Inteiro, com valor maior ou igual a um.
Desconto do item	Decimal128, maior ou igual a zero e menor que um.
Datas	BSON Date; data de envio admite null.
Endereço de envio	Objeto obrigatório, com destinatário, logradouro, cidade, região, CEP e país; região e CEP admitem null.
Itens do pedido	Array de objetos com produto, categoria, preço praticado, quantidade e desconto.

O trecho a seguir apresenta a regra aplicada ao desconto dos itens, incluindo tipo, limites e exclusão do limite superior.

```
discount: {
  bsonType: "decimal",
  minimum: Decimal128("0"),
  maximum: Decimal128("1"),
  exclusiveMaximum: true
}
```

8.6 Testes de rejeição

Foram realizadas cinco tentativas de substituição de um pedido válido em uma base temporária isolada. Cada tentativa alterou apenas o aspecto indicado na tabela.

Alteração testada	Resultado
Preço armazenado como texto	Rejeitada: código 121.
Quantidade igual a zero	Rejeitada: código 121.
Desconto igual a 100%	Rejeitada: código 121.
Data do pedido armazenada como texto	Rejeitada: código 121.
Ausência do objeto de endereço de envio	Rejeitada: código 121.

Todas as alterações foram rejeitadas, e o documento válido permaneceu inalterado. O resultado demonstra a atuação dos validadores diante das inconsistências testadas, além da constatação de que os documentos carregados atendiam às regras.

Os validadores não substituem as chaves estrangeiras nem reproduzem todas as restrições do modelo relacional. A existência dos cadastros referenciados, a coerência temporal e a ausência de produtos repetidos dentro de um pedido foram verificadas separadamente, após a carga.

9. Estratégia de indexação

9.1 Índices associados a consultas

Além do índice único automático em `_id` de cada coleção, foram criados três índices adicionais. Eles atendem a formas concretas de acesso previstas para o modelo.

Índice	Campos	Consulta atendida
cliente_data	customer_id: 1, order_date: 1	Pedidos de um cliente em determinado período, ordenados por data.
produto_no_pedido	items.product._id: 1	Pedidos que contêm determinado produto.
categoria_produto	category._id: 1	Produtos cadastrados em uma categoria.

No índice composto, o cliente é usado por igualdade e a data permite o intervalo e a ordenação. O índice em `items.product._id` é **multikey**: o MongoDB cria entradas para valores encontrados no array de itens [10]. Não foi usado índice único nesse campo, porque o mesmo produto pode aparecer em vários pedidos.

```
db.orders.createIndex(
  { customer_id: 1, order_date: 1 },
  { name: "cliente_data" }
);
```

9.2 Evidência do plano de execução

A consulta abaixo foi executada com `explain`, sem forçar índice com `hint`:

```
db.orders.find({
  customer_id: "VINET",
  order_date: {
    $gte: ISODate("1996-01-01T00:00:00Z"),
    $lt: ISODate("1997-01-01T00:00:00Z")
  }
}).sort({ order_date: 1 }).explain("executionStats");
```

Medida observada	Resultado
Plano vencedor	FETCH → IXSCAN
Índice utilizado	cliente_data
Documentos retornados	3
Documentos examinados	3
Chaves examinadas	3

IXSCAN indica leitura pelo índice; **FETCH** indica a busca dos documentos correspondentes. A consulta retorna três pedidos de VINET em 1996. Esse resultado demonstra uso do índice nessa consulta, sem justificar uma conclusão geral sobre superioridade do MongoDB.

Índices consomem espaço e precisam ser mantidos durante as escritas. Por isso, não foram criados índices para todos os campos. A observação realizada sustenta o uso do índice composto para o filtro e a ordenação examinados. Os outros dois índices foram definidos pelos acessos previstos, mas seus planos de execução não foram avaliados nesta análise. A comparação de desempenho entre bancos permanece em aberto.

10. Conferência da migração

10.1 Equivalência dos registros

A conferência foi realizada em 10/09/2026, mediante a leitura dos documentos gravados no MongoDB e a reconstrução das linhas das 11 tabelas de nw. Os itens foram separados dos pedidos, os campos dos endereços foram desagrupados e os vínculos entre funcionários e territórios foram reconstituídos. Esse procedimento foi desenvolvido separadamente da transformação utilizada na carga.

As linhas reconstruídas foram comparadas com a origem por **EXCEPT ALL nos dois sentidos**. A comparação abrangeu os campos e as ocorrências de cada linha, permitindo identificar registros ausentes, extras, duplicados ou com valores alterados. Os decimais foram comparados sem arredondamento prévio, e datas com horário diferente da convenção de meia-noite UTC foram recusadas antes da comparação.

Tabela reconstruída	Origem	MongoDB	Diferenças
regions	4	4	0
territories	53	53	0
categories	8	8	0
shippers	6	6	0
suppliers	29	29	0
customers	91	91	0
employees	9	9	0
employee_territories	49	49	0
products	77	77	0
orders	830	830	0
order_items	2.155	2.155	0
Total	3.311	3.311	0

As **3.311 linhas** foram reconstruídas sem divergências. Também foram conferidos os tipos BSON, as referências entre coleções, os nomes e categorias copiados para os itens, os territórios incorporados e a coerência das datas. A comparação foi repetida após a segunda carga, mantendo zero diferenças.

10.2 Precisão dos valores

A soma dos itens foi calculada separadamente nos dois bancos pela expressão **preço praticado × quantidade × (1 – desconto)**, sem inclusão do frete. PostgreSQL e MongoDB retornaram o mesmo valor decimal exato: **1265793.03950**. Com arredondamento apenas após a soma, o valor apresentado é **1.265.793,04**.

A conferência não utilizou as views que arredondam cada item. A comparação exata também evita depender das diferenças entre round(numeric) do PostgreSQL e \$round do MongoDB em casos de empate [4, 11].

Como teste adicional, uma cópia dos dados de retorno recebeu o preço 14.004 em lugar de 14.00. A comparação identificou duas diferenças: a ausência da linha original e a presença da linha alterada. O teste confirmou que uma diferença inferior a um centavo não foi ocultada por arredondamento. Nenhum registro dos bancos foi alterado nesse teste.

11. Conclusões e continuidade

A modelagem documental representou os dados relacionais do Northwind em nove coleções, com 1.107 documentos. A incorporação dos itens e do endereço de envio permitiu reunir os dados de cada pedido em uma única estrutura. Os cadastros permaneceram em coleções próprias, mantendo as referências necessárias e os registros sem movimento.

A transformação preservou os identificadores, os campos nulos, as datas, os preços praticados, os descontos e os endereços dos pedidos. A conferência independente reconstruiu as 3.311 linhas de nw sem divergências. A repetição da carga não inseriu nem modificou documentos, confirmando que o mesmo conjunto de dados pode ser carregado novamente sem duplicação.

11.1 Avaliação do modelo

O pedido 10248 exemplifica a organização adotada: seus três itens, antes armazenados em linhas separadas, passaram a integrar um array no documento. Produto e categoria foram representados em objetos aninhados, permitindo identificar os itens sem consultar outros cadastros para obter seus nomes. O custo dessa escolha é a manutenção das informações copiadas quando os cadastros mudam.

Os validadores rejeitaram as cinco alterações inválidas testadas e preservaram o documento original. A existência das referências e a coerência entre documentos foram verificadas separadamente, pois a estrutura documental não oferece as mesmas garantias das chaves estrangeiras relacionais.

Os índices foram definidos para pedidos por cliente e período, pedidos por produto e produtos por categoria. Na consulta observada para o cliente VINET em 1996, o MongoDB utilizou o índice composto e examinou três documentos para retornar três pedidos. Esse resultado sustenta a escolha do índice para o acesso avaliado.

11.2 Limitações e continuidade

Os resultados comprovam a preservação dos dados no conjunto analisado, mas não permitem concluir sobre escalabilidade ou superioridade de desempenho entre os bancos. O volume é pequeno, e as medições de tempo de consultas equivalentes ainda não foram realizadas.

A carga em lote depende de nova extração para refletir mudanças na origem. As cópias de nomes e categorias representam o estado cadastral disponível na extração, sem reconstruir seu histórico na data da venda. O modo standalone também limita a atomicidade ao documento, exigindo nova conferência após a recuperação de uma carga interrompida.

A continuidade do trabalho consiste na implementação de consultas analíticas equivalentes e na avaliação de desempenho, mantendo as definições dos indicadores, o preço praticado no item e os critérios de precisão. A equivalência já verificada fornece uma base consistente para comparar os dois modelos sem atribuir a diferenças de tecnologia resultados causados por perda ou alteração de dados.

Referências

- [1] IBM. **CRISP-DM Help Overview**. IBM SPSS Modeler. Orientação sobre as fases de compreensão do negócio e dos dados. Disponível em: <https://www.ibm.com/docs/en/spss-modeler/18.6.0?topic=dm-crisp-help-overview>. Acesso em: 5 set. 2026.
- [2] PTHOM. **Northwind sample database for PostgreSQL**. Repositório da adaptação utilizada neste projeto. Disponível em: https://github.com/pthom/northwind_psql. Acesso em: 5 set. 2026.
- [3] IPOG. **Livro Digital — Projeto Integrador**. Unidade 4: Projeto Área 03 — Banco de Dados, seções 4.1, 4.2 e 4.5.2 a 4.5.4. Material didático da disciplina.
- [4] POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation: Numeric Types**. Disponível em: <https://www.postgresql.org/docs/16/datatype-numeric.html>. Acesso em: 5 set. 2026.
- [5] MONGODB. **Best Practices for Data Modeling in MongoDB**. Disponível em: <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>. Acesso em: 5 set. 2026.
- [6] MONGODB. **Model Monetary Data**. Disponível em: <https://www.mongodb.com/docs/manual/tutorial/model-monetary-data/>. Acesso em: 5 set. 2026.
- [7] MONGODB. **Schema Validation**. Disponível em: <https://www.mongodb.com/docs/manual/core/schema-validation/>. Acesso em: 5 set. 2026.
- [8] MONGODB. **Production Considerations: Transactions**. Disponível em: <https://www.mongodb.com/docs/manual/core/transactions-production-consideration/>. Acesso em: 5 set. 2026.
- [9] MONGODB. **MongoDB Extended JSON (v2)**. Disponível em: <https://www.mongodb.com/docs/manual/reference/mongodb-extended-json/>. Acesso em: 10 set. 2026.
- [10] MONGODB. **Multikey Indexes**. Disponível em: <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-multikey/>. Acesso em: 10 set. 2026.
- [11] MONGODB. **\$round (expression operator)**. Disponível em: <https://www.mongodb.com/docs/manual/reference/operator/aggregation/round/>. Acesso em: 10 set. 2026.
- [12] POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16: JSON Functions and Operators**. Disponível em: <https://www.postgresql.org/docs/16/functions-json.html>. Acesso em: 10 set. 2026.